

Lowest Common Ancestor and Binary Lifting

Sean

September 30, 2026



Outline

- Lowest common ancestor
- Binary lifting
- Answering LCA queries
- Distances and applications



Why ask about ancestors?

- The first tree lecture gave us parents and depths with one DFS
- Now we want to answer many questions about pairs of vertices
- The branching point of their paths to the root is the key



1 Lowest common ancestor

2 Binary lifting

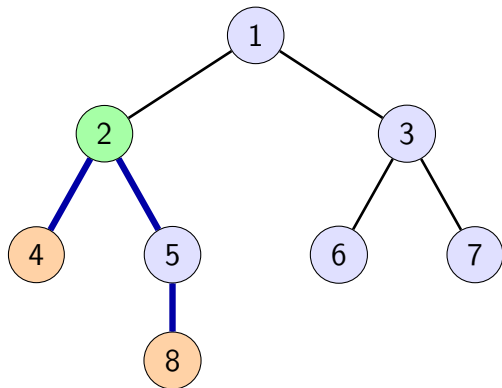
3 Answering LCA queries

4 Distances and applications



Lowest common ancestor

Where do two paths meet?



- Root at 1
 - Paths from 4 and 8 meet at 2
- $LCA(4, 8) = 2$

What does LCA mean?

- The lowest common ancestor of u and v is their deepest shared ancestor
- A vertex is its own ancestor, so $LCA(u, u) = u$
- In the example, $LCA(8, 7) = 1$ and $LCA(8, 5) = 5$
- The answer depends on which vertex we choose as the root



- 1 Lowest common ancestor
- 2 Binary lifting
- 3 Answering LCA queries
- 4 Distances and applications

Binary lifting

Jump by powers of two

- Following one parent edge per step is slow for a deep tree
- Store the 1st, 2nd, 4th, 8th, ... ancestor of each vertex
- Any upward move can be assembled from these jumps

$13 = 8 + 4 + 1 \quad \implies \quad$ three jumps instead of thirteen

The jump table

Meaning of $\text{up}[v][j]$

The ancestor 2^j edges above v ; use 0 if that ancestor does not exist.

$$\begin{aligned}\text{up}[v][0] &= \text{parent}[v], \\ \text{up}[v][j] &= \text{up}[\text{up}[v][j-1]][j-1].\end{aligned}$$

- Set $\text{parent}[\text{root}] = 0$ and every $\text{up}[0][j] = 0$
- Compute parents and depths with DFS, then fill higher jump levels



Example: ancestors of vertex 8

j	Jump length 2^j	$\text{up}[8][j]$	Path from 8
0	1	5	$8 \rightarrow 5$
1	2	2	$8 \rightarrow 5 \rightarrow 2$
2	4	0	above the root

$$\text{up}[8][1] = \text{up}[\text{up}[8][0]][0] = \text{up}[5][0] = 2$$

- To climb 3 edges from 8, jump 2 edges to 2, then 1 to 1



Build the jump table in C++

A DFS or BFS from root 1 gives `parent[v]` and `depth[v]`;
set `parent[1] = 0`.

```
int LOG = 1;
while ((1LL << LOG) <= n) ++LOG;
vector<vector<int>> up(n + 1, vector<int>(LOG));

for (int v = 1; v <= n; ++v) up[v][0] = parent[v];
for (int j = 1; j < LOG; ++j)
    for (int v = 1; v <= n; ++v)
        up[v][j] = up[up[v][j - 1]][j - 1];
```

Vertex 0 is the sentinel; its jump row stays zero.



1 Lowest common ancestor

2 Binary lifting

3 Answering LCA queries

4 Distances and applications



Answering LCA queries

Two steps to find the LCA

- First use binary jumps to bring the deeper vertex to the same depth
- If they are now equal, that vertex is the answer
- Otherwise, try jump sizes from largest to smallest
- Move both vertices only when their jump destinations differ
- Their shared parent is the LCA



Trace: $\text{LCA}(8, 7)$

Step	Vertices	Reason
Start	8 at depth 3, 7 at depth 2	Unequal depths
Lift	$8 \rightarrow 5$, 7 stays	Both at depth 2
Compare	$5 \rightarrow 2$, $7 \rightarrow 3$	Parents differ; move both
Finish	2 and 3	Shared parent is 1

$$\text{LCA}(8, 7) = 1$$



LCA query in C++

Define this lambda after building up; it uses the same depth array.

```
auto lca = [&](int u, int v) -> int {  
    if (depth[u] < depth[v]) swap(u, v);  
    int gap = depth[u] - depth[v];  
    for (int j = LOG - 1; j >= 0; --j)  
        if (gap & (1LL << j)) u = up[u][j];  
  
    if (u == v) return u;  
    for (int j = LOG - 1; j >= 0; --j)  
        if (up[u][j] != up[v][j]) {  
            u = up[u][j];  
            v = up[v][j];  
        }  
    return up[u][0];  
};
```



- 1 Lowest common ancestor
- 2 Binary lifting
- 3 Answering LCA queries
- 4 Distances and applications



Distances and applications

Distance between two vertices

- The unique path from u to v goes through their LCA
- Count the edges from u up, then from the LCA down to v

$$\text{dist}(u, v) = \text{depth}(u) + \text{depth}(v) - 2 \text{depth}(\text{LCA}(u, v)).$$

Example: $\text{dist}(8, 7) = 3 + 2 - 2(0) = 5$ edges.



When is binary lifting useful?

- Find the k th ancestor of a vertex
- Find the LCA of two vertices
- Answer distances between vertices using their LCA and depths

Costs

$O(n \log n)$ preprocessing and memory; $O(\log n)$ per query.



Watch for these mistakes

- Mixing up the 2^j jump level with a jump of j edges
- Forgetting to align depths before moving both vertices
- Reading past the root instead of using a safe sentinel
- Counting vertices when the problem asks for path length in edges



Practice roadmap

- CSES: Company Queries I — k th ancestor queries
- CSES: Company Queries II — LCA queries
- CSES: Distance Queries — combine LCA with depths



The End

Thanks for listening.

© 2026 Sean. Please do not repost or redistribute without permission.

